



PYTHON NOTES — DECISION MAKING



TOPIC: DECISION MAKING

Decision making allows a Python program to choose what to do based on whether a condition is `True` or `False`.

This is what makes programs able to respond differently to different situations.

1. BOOLEAN VALUES

A Boolean value can be either:

`True`

or:

`False`

Booleans are commonly produced when Python compares values.

Example:

```
print(5 > 3)
```

Output:

`True`

Another example:

```
print(2 > 8)
```

Output:

False

2. COMPARISON OPERATORS

Comparison operators are used to compare values.

Equal to `==`

5 == 5

Result:

True

Not equal to `!=`

5 != 3

Result:

True

Greater than `>`

8 > 5

Result:

True

Less than `<`

3 < 5

Result:

True

Greater than or equal to `>=`

`70 >= 70`

Result:

True

Less than or equal to `<=`

`50 <= 70`

Result:

True

These operators are especially useful with `if`, `elif`, and `else`.

3. `if` STATEMENTS

An `if` statement tells Python to execute a block of code **only when a condition is true**.

Example:

```
age = 18

if age >= 18:
    print('You are 18 or older.')
```

Because `18 >= 18` is `True`, Python prints the message.

Basic structure

```
if condition:
    # code to run when the condition is True
```

The colon `:` is important, and the code inside the `if` block must be indented.

4. `else`

`else` is used when we want Python to do something when the `if` condition is false.

Example:

```
age = 15

if age >= 18:
    print('Adult')
else:
    print('Not an adult')
```

Since `15 >= 18` is `False`, Python runs the `else` block.

Structure

```
if condition:
    # runs if True
else:
    # runs if False
```

5. `elif`

`elif` means **else if**.

It allows us to check another condition when the previous `if` condition was false.

Example:

```
score = 65

if score >= 70:
    print('Excellent!')
elif score >= 50:
    print('Good')
else:
    print('You need to improve')
```

Python checks the conditions from top to bottom.

For `score = 65`:

```
65 >= 70 → False
65 >= 50 → True
```

So Python prints:

```
Good
```

6. `if`, `elif`, AND `else` TOGETHER

A common decision-making structure is:

```
if condition1:
    # first choice
elif condition2:
    # second choice
else:
    # final choice
```

Python checks the conditions in order and runs the first matching block.

7. LOGICAL OPERATORS

Logical operators allow us to combine or change conditions.

The main logical operators are:

- `and`
 - `or`
 - `not`
-

8. `and`

`and` requires **both conditions** to be true.

Example:

```
age = 20
score = 80

if age >= 18 and score >= 70:
    print('Both conditions are true.')
```

Both conditions must be `True` for the `if` block to run.

Think:

Condition A AND Condition B

Both must be true.

9. `or`

`or` requires **at least one condition** to be true.

Example:

```
score = 80

if score >= 70 or score == 50:
    print('The condition is true.')
```

Only one of the conditions needs to be true.

Think:

Condition A OR Condition B

At least one must be true.

10. not

`not` reverses a Boolean result.

Example:

```
is_raining = False

if not is_raining:
    print('It is not raining.')
```

`not False` becomes `True`.

11. NESTED `if` STATEMENTS

A nested `if` statement is an `if` statement inside another `if` statement.

Example:

```
age = 18

if age >= 18:
    print('You are an adult.')

    if age >= 21:
        print('You are 21 or older.')
```

The second `if` only gets checked after the first condition is true.

12. USING `input()` WITH DECISION MAKING

Decision-making programs often get information from the user.

Example:

```
age = int(input('Enter your age: '))

if age >= 18:
    print('You are an adult.')
else:
    print('You are under 18.')
```

The flow is:

```
input()
↓
convert input if necessary
↓
store in variable
↓
compare value
↓
make a decision
```

13. MY EXAM SCORE CHECKER PROGRAM

This is a program I created while learning Decision Making.

```
name = input('What is your name?')
age = int(input('Type in your age'))
hobby = input('What is your favorite hobby?')
score = int(input('Give me your exam score'))

if score >=70:
    print("Excellent!")
elif score >=50:
    print("Good")
else:
    print("You need to improve")
```

```
print('My Name is', name ,',I am', age , 'years old. I love', hobby, 'and my  
exam score checker told me', score)
```

What happens in the program?

First, the program asks the user for:

- Name
- Age
- Favorite hobby
- Exam score

The age and score are converted into integers using `int()`.

Then the program checks the exam score.

If the score is 70 or more

```
if score >= 70:  
    print('Excellent!')
```

The program prints:

```
Excellent!
```

If the score is below 70 but at least 50

```
elif score >= 50:  
    print('Good')
```

The program prints:

```
Good
```

If neither condition is true

```
else:  
    print('You need to improve')
```

The program prints:

You need to improve

This is a good example of using **Python Basics + Decision Making** together.

14. ORDER OF CONDITIONS MATTERS

Python checks `if` and `elif` conditions from top to bottom.

Example:

```
score = 90

if score >= 50:
    print('Good')
elif score >= 70:
    print('Excellent!')
```

This would print:

Good

Why?

Because `90 >= 50` is already true, so Python runs that block and does not continue to the next `elif`.

Therefore, when using ranges, we need to arrange our conditions carefully.

A better version is:

```
if score >= 70:
    print('Excellent!')
elif score >= 50:
    print('Good')
else:
    print('You need to improve')
```

15. F-STRING VERSION OF MY PROGRAM

I can reuse my knowledge of f-strings from the Functions topic to make the final message cleaner.

```
print(f'My name is {name}. I am {age} years old. I love {hobby}, and my exam  
score was {score}.')
```

This is an example of combining **new knowledge with knowledge learned earlier**.

The f-string itself is not a Decision Making concept; it is simply being reused here.

WHAT I CAN DO NOW

After learning Decision Making, I can:

- ✓ Understand `True` and `False`
 - ✓ Compare values using comparison operators
 - ✓ Use `if` statements
 - ✓ Use `elif` statements
 - ✓ Use `else` statements
 - ✓ Combine conditions with `and`
 - ✓ Combine conditions with `or`
 - ✓ Reverse conditions with `not`
 - ✓ Create nested `if` statements
 - ✓ Use `input()` in decision-making programs
 - ✓ Convert user input using `int()` before making decisions
 - ✓ Build programs that respond differently to different values
 - ✓ Combine Decision Making with Python Basics and previously learned concepts
-

MY PROGRESS

Current Level: Decision Making ✓

Topics Learned

- ✓ Boolean values
- ✓ `True` and `False`
- ✓ `if`
- ✓ `elif`
- ✓ `else`

- ✓ `and`
 - ✓ `or`
 - ✓ `not`
 - ✓ Nested `if` statements
 - ✓ Combining `input()` with decisions
 - ✓ Combining Decision Making with Python Basics
-

DECISION MAKING COMPLETE

Decision Making taught me how to make my programs respond differently depending on the information they receive.

I can now create programs that make choices instead of always following exactly the same path.

 **DECISION MAKING → ✓ COMPLETE**

Next major topic: LOOPS 